

Mini-JARVIS: informe tecnico

Asistente conversacional por voz con exploracion de la arquitectura Transformer

Proyecto integrador de Redes Neuronales Desarrollo de Software, CENESTUR Britany Macias, agosto de 2026
Repositorio: <https://github.com/TahisMacias/proyecto-minijarvis>

1. Que hace

Mini-JARVIS es un asistente de voz en espanol que corre como aplicacion de escritorio en Windows. Mantengo presionado un boton, hablo, y la asistente transcribe lo que dije, piensa una respuesta con un modelo de lenguaje y me la contesta en voz alta. Se acuerda de lo que hablamos en turnos anteriores.

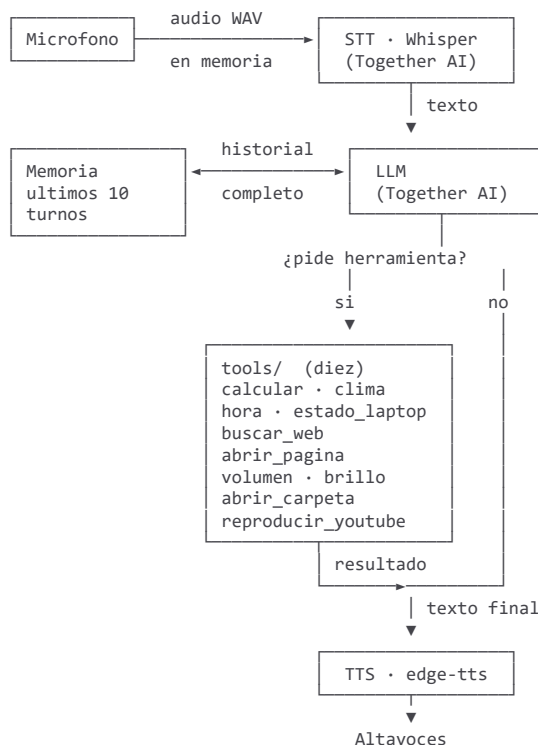
La asistente se llama Elena. Mini-JARVIS es el nombre del proyecto, pero a ella la llamo por su nombre y responde cuando lo digo. La seccion 4 del enunciado deja la personalidad a criterio de cada equipo si queda definida en un system prompt documentado, y asi esta: el nombre vive en una sola constante de `config.py`, de donde lo leen la ventana, el saludo y los dos system prompts.

Ademas de conversar usa diez herramientas. Resuelve cuentas exactas, dice la hora, consulta el clima y el estado de la laptop, busca en internet, abre paginas de una lista autorizada, sube y baja el volumen y el brillo, abre carpetas y pone musica en YouTube.

Y en la misma ventana, junto a la conversacion, se ve que le pasa por dentro a un Transformer con la ultima frase que dije: en que pedazos se corta, que numero le toca a cada uno y a que otras palabras presta atencion cada una. Ese es el motivo de que el proyecto tenga dos modelos y no uno, y lo explico en la seccion 5.

2. Arquitectura

2.1 El recorrido de un turno



Todo lo coordina `core/orchestrator.py`, que mantiene la maquina de estados y le avisa a la interfaz de cada cambio.

El laboratorio del Transformer corre aparte, en paralelo, sobre la frase transcrita. No forma parte de este recorrido.

2.2 Los modulos

Archivo	Responsabilidad
<code>config.py</code>	La unica fuente de configuracion: credenciales, paleta, identificadores de modelo, limites. Ningun otro modulo lee variables de entorno.
<code>core/audio_capture.py</code>	Microfono a WAV en memoria, con push-to-talk
<code>core/stt_client.py</code>	WAV a texto
<code>core/memory.py</code>	Historial de la conversacion y su recorte
<code>core/llm_engine.py</code>	Mensajes a respuesta del modelo. No ejecuta herramientas
<code>core/tts_engine.py</code>	Texto a voz reproducida
<code>core/orchestrator.py</code>	Maquina de estados y el hilo de cada turno
<code>core/modo_local.py</code>	Respaldo para cuando no hay internet
<code>tools/manifest.py</code>	Lo que el modelo lee para saber que puede pedir
<code>tools/system_skills.py</code>	Lo que las herramientas hacen
<code>gui/desktop_app.py</code>	La ventana
<code>exploration/transformer_lab.py</code>	El laboratorio, independiente del resto

3. La teoria, con la salida real del programa

Todo lo de esta seccion sale de ejecutar `python -m exploration.transformer_lab`. No son cifras de libro: son las que imprime mi proyecto, y estan guardadas en `docs/evidencia/T-11-salida-transformer_lab.txt`.

3.1 Tokenizacion

Antes de que un texto entre a la red neuronal, un componente llamado tokenizador lo corta en fragmentos de un vocabulario fijo y traduce cada uno a un numero entero. El modelo, por dentro, solo ve una lista de numeros.

Con el tokenizador real de Qwen2.5 y una frase propia del proyecto, la salida da 30 tokens. Lo interesante es lo que pasa con las palabras que no estan en el vocabulario: se reconstruyen pegando varios sub-tokens. "Mini-JARVIS" no aparecio en los datos de entrenamiento, asi que se parte. Las palabras acentuadas se parten mas que las demas.

Ese detalle tiene una explicacion concreta. El tokenizador de Qwen es byte-level BPE, es decir, no trabaja sobre letras sino sobre los bytes del texto en UTF-8. Una letra acentuada ocupa dos bytes, asi que aparece partida y con simbolos de aspecto extrano en pantalla. No es un error de codificacion. Es la representacion interna real del modelo, y por eso el programa lo advierte en su salida: sin ese aviso parece que algo esta roto.

3.2 Embeddings

Un embedding es un vector que representa el significado de un token dentro de su contexto. En BERT cada token se representa con 768 numeros.

Salida real del programa, para una frase de 18 sub-tokens mas las dos marcas `[CLS]` y `[SEP]`:

Forma del tensor: (1, 20, 768)

El 1 es que proceso una sola frase a la vez. El 20 son los tokens de esa frase, marcas incluidas. El 768 es el tamaño del vector de significado de cada token.

No hay una etiqueta humana para cada uno de esos 768 números. Lo que importa es que tokens con usos parecidos acaban con vectores parecidos, en un espacio que el modelo aprendió durante su entrenamiento.

3.3 Positional encoding

El mecanismo de atención, por sí solo, no tiene noción de orden. Cada token mira a todos los demás a la vez, así que para el una frase es un conjunto de palabras y no una secuencia. Si nada más interviniera, "el perro mordió al gato" y "el gato mordió al perro" serían la misma entrada.

La solución es sumarle a cada token, antes de entrar a las capas de atención, un segundo vector que depende solo de la posición que ocupa. BETO guarda esos vectores en una tabla aparte:

```
Tabla de palabras : (31002, 768)
Tabla de posiciones: (512, 768)
```

De ahí sale, de paso, por que el modelo no admite frases de más de 512 tokens: no tiene vector de posición para el número 513.

Dos cosas más que compruebo con el programa. La primera: la misma palabra en dos posiciones distintas produce vectores distintos. Comparando "perro" en la posición 1 y en la 2, el parecido es 0.9367 y no 1.0000, que es lo que saldría si la posición no influyera. El script se detiene con un error si alguna vez diera 1.0, porque eso significaría que el positional encoding no se está aplicando.

La segunda: ese parecido decae con la distancia. Entre posiciones vecinas es 0.902, a tres de distancia baja a 0.513, y a cinco a 0.394. El modelo no solo sabe donde está cada palabra, sabe cuanto se separan entre sí. La imagen [exploration/mapa_posiciones.png](#) lo muestra como una diagonal brillante que se apaga hacia los lados.

Un apunte que conviene saber: en el artículo original del Transformer, de 2017, las posiciones se calculaban con senos y cosenos. BETO, como todos los BERT, las aprende igual que aprende las palabras. Las dos formas resuelven el mismo problema.

3.4 Self-attention

Es el mecanismo central del Transformer. Cada token mira a todos los demás de la frase, incluido el mismo, y decide cuanto peso darle a cada uno para construir su propio significado contextual.

Salida real:

```
Numero de capas de atencion devueltas: 12 (BETO tiene 12 capas Transformer).
Forma de UNA capa de atencion: (1, 12, 20, 20)
Suma de esa fila: 1.000000
```

Las cuatro dimensiones son: una frase, 12 cabezas de atención, y una matriz de 20 por 20 donde la fila es el token que consulta y la columna el token consultado.

Que la fila sume exactamente 1.0 no es casualidad ni un ajuste. Cada fila es el resultado de una función softmax, que convierte una lista de puntajes en una distribución de probabilidad. Significa que cada token reparte el cien por cien de su atención entre los demás, ni más ni menos. El programa lo verifica en cada ejecución y se detiene con un error si dejara de cumplirse.

3.5 Capas, cabezas y el mapa de calor

Un Transformer no procesa la frase de una pasada. La recorre por capas, doce en BETO, y cada capa entiende un poco mas que la anterior. Dentro de cada capa hay doce cabezas mirando en paralelo, y cada una puede especializarse en un tipo de relacion distinto.

Probe las 144 combinaciones de capa por cabeza midiendo cuanta atencion pone cada una en palabras de contenido en vez de en las marcas [CLS] y [SEP]. La capa 6, cabeza 4, resulto ser una cabeza de token anterior: cada token le presta casi toda su atencion a la palabra que lo precede. En el mapa se ve como una franja iluminada justo a la izquierda de la diagonal principal. Es un patron clasico y muy documentado en modelos tipo BERT, y se reconoce de un vistazo incluso proyectado.

Imagen: [exploration/mapa_atencion.png](#).

3.6 Encoder-only y decoder-only: uso los dos

El enunciado pide poder explicar por que los LLM conversacionales usan arquitectura decoder-only. Mi proyecto es un buen sitio para verlo porque tiene un modelo de cada tipo funcionando a la vez.

BETO, el del laboratorio, es encoder-only. Su trabajo es leer: recibe la frase entera de golpe y construye una representacion de cada palabra mirando a las de su izquierda y a las de su derecha. Por eso se le pueden pedir los embeddings y las matrices de atencion de toda la frase, porque ya la vio completa. No sirve para conversar: no esta hecho para producir texto nuevo.

Qwen, el que responde en la aplicacion, es decoder-only. Su trabajo es escribir. Genera una palabra, la anade a lo que lleva escrito, y con eso genera la siguiente. A eso se le llama autoregresivo. Cada palabra solo puede mirar hacia atras, nunca hacia adelante, porque lo que va delante todavia no existe.

Ahi esta el motivo de que los asistentes usen decoder-only. Conversar es generar texto, y generar texto es exactamente lo que hace un decoder. Un encoder entiende muy bien y no produce nada; un decoder produce, y para eso tiene que entender lo suficiente.

Es tambien la razon de la limitacion de la seccion 5: puedo mirar por dentro a BETO porque corre aqui, y no a Qwen porque corre en un servidor ajeno.

3.7 Preentrenamiento, fine-tuning e instruction-tuning

Son tres etapas distintas.

En el preentrenamiento el modelo lee cantidades enormes de texto y aprende una sola tarea: predecir la palabra siguiente. De ahi sale un modelo base, que sabe muchisimo del lenguaje y no sabe conversar. Si a un modelo base le escribes "hola, quien eres", lo mas probable es que continue el texto, inventando un dialogo entero o una lista de preguntas parecidas, en vez de contestarte. Continuar texto es literalmente lo unico que le ensenaron.

El fine-tuning es seguir entrenando ese modelo base sobre un conjunto de datos mas pequeno y especifico, para especializarlo en un dominio o un estilo.

El instruction-tuning es el fine-tuning concreto que convierte un modelo base en uno de chat: se le entrena con ejemplos de instruccion y respuesta hasta que aprende que, cuando le llega algo que parece una pregunta, lo que toca es responderla.

Los dos modelos de mi proyecto pasaron por esa etapa, y se nota en el nombre: [Qwen2.5-0.5B-Instruct](#) y [Llama-3.3-70B-Instruct-Turbo](#) llevan "Instruct" justo por eso. Sin instruction-tuning, el system prompt que define la personalidad de Elena no serviria de nada, porque un modelo base no obedece instrucciones: las continua.

4. Decisiones de diseño, con lo que descarte

4.1 Un hilo efímero por turno, en vez de asyncio

Descarte un bucle `asyncio` de larga vida, que era la propuesta inicial.

Tkinter obliga a que su `mainloop` viva en el hilo principal. Sostener en paralelo un segundo modelo de concurrencia es donde se pierden días depurando ventanas congeladas. Elegí un hilo trabajador que nace cuando suelto el botón y muere cuando la respuesta terminó de sonar.

La regla se protege sola: el orquestador no importa nada de la interfaz. Su única salida es una función `notificar(evento)` que la ventana le entrega, y que se limita a reenviar el evento con `root.after(0, ...)`, la única forma segura de cruzar de un hilo cualquiera al de la interfaz. Hay una prueba que lee el código del orquestador y falla si alguna vez apareciera ahí un `import` de Tkinter.

4.2 La calculadora no usa eval

Descarte `eval(expresion)`, que son tres caracteres y funciona.

`eval` no distingue una suma de una llamada al sistema operativo, y la expresión la escribe un modelo de lenguaje, no una persona de confianza.

Lo que hice fue analizar la expresión con el módulo `ast`, rechazar cualquier elemento que no esté en una lista blanca de operaciones y funciones matemáticas, y calcular el resultado recorriendo el árbol a mano. No hay `eval`, `exec` ni `compile` en todo `tools/`, y una prueba lee el código de cada archivo y falla si alguien los anade.

4.3 Truncar la memoria por turnos, no por tokens

Descarte recortar el historial contando tokens, que aprovecha mejor la ventana de contexto.

El resultado dependería del tokenizador y sería difícil de predecir y de verificar. Truncar por turnos es predecible, se puede enseñar en vivo y se prueba entero sin llamar a ninguna API.

4.4 Dos modelos de familias distintas

El selector ofrece un modelo de razonamiento grande y uno convencional de otra familia. Uno escribe un borrador interno antes de contestar y el otro no, y la diferencia se nota en la latencia y en la longitud de la respuesta. Dos tamaños del mismo modelo no enseñarían nada.

4.5 Listas blancas para todo lo que toca el sistema

Las páginas que puede abrir y las carpetas a las que llega salen de listas cerradas, no de lo que diga el modelo. Si la transcripción sale mal, lo peor que pasa es que no encuentre la carpeta.

5. La limitación más importante

No puedo inspeccionar la atención del modelo que responde en la aplicación.

Los modelos de Together AI corren en sus servidores, detrás de un endpoint HTTP que solo devuelve texto. Pedirle a ese endpoint sus matrices internas de self-attention es físicamente imposible: el modelo no está en esta computadora.

Por eso el laboratorio usa BETO (`dccuchile/bert-base-spanish-wwm-cased`, unos 110 millones de parámetros, entrenado en español), que si se descarga y se ejecuta entero en la máquina local. Es un Transformer real, y como está en mi memoria lo puedo abrir y mirar por dentro.

Es una limitación honesta del enfoque, no un atajo. La alternativa habría sido no poder demostrar self-attention en absoluto.

5.1 Una trampa que casi cuesta el modulo entero

A partir de `transformers 5.x`, el backend de atencion por defecto es SDPA, que no materializa las matrices intermedias. Si se pide `output_attentions=True` con SDPA, el resultado llega vacío y no se lanza ninguna excepcion. El programa corre bien y no cumple su objetivo.

La correccion es cargar el modelo con `attn_implementation="eager"`. El laboratorio además lo comprueba en tiempo de ejecucion y se detiene con un mensaje explicito si volviera a ocurrir.

6. Otras limitaciones conocidas

La conversacion no sobrevive al cierre. La memoria vive en RAM y al cerrar la aplicacion se pierde. Fue una decision de alcance: no hay base de datos.

La disponibilidad de un modelo caduca. De los 169 modelos de chat que lista Together, solo 20 responden; el resto devuelve `HTTP 400 non-serverless`. El campo `running` del catalogo no sirve para saberlo, porque viene en `false` para todos. Y hay algo peor: el modelo alterno que elegi el 14 de agosto, verificado entonces contra la API, devolvía `HTTP 503` tres días después. Haber probado un modelo no lo garantiza para siempre.

La temperatura tiene un tope de 1.4. Midiendo contra la API, a partir de 1.5 el modelo de razonamiento se atasca unos 100 segundos en dos de cada tres intentos.

Sin internet la aplicacion sigue funcionando, pero responde bastante peor. El modelo local tiene 494 millones de parametros frente a los billones del de la nube. Preguntandole dos veces la capital de Ecuador contesto "Quito" una vez y "Santo Domingo" la otra. Sirve para que la aplicacion siga viva, no para confiar en lo que dice, y la propia aplicacion lo advierte cuando entra ese modo.

Un microfono que capta solo ruido no es un fallo detectable: produce una transcripcion equivocada, no vacia, y se atiende como conversacion normal.

Solo funciona en Windows. La reproduccion de voz usa la interfaz multimedia del sistema.

7. Manejo de errores

El diseno preveia siete fallos y los siete estan cubiertos con un mensaje propio, redactado para una persona y sin trazas tecnicas: microfono no disponible, sin conexion, credenciales invalidas, transcripcion vacia, respuesta vacia del modelo (con un reintento unico), JSON de herramienta malformado y fallo de la sintesis de voz.

La regla que los une es que cualquier excepcion que no venga de un modulo del proyecto se reemplaza por un mensaje generico, para que una traza tecnica no llegue nunca a la pantalla. Y todo camino de fallo termina igual:

```
... -> ATENCION -> REPOSO
```

Nunca se queda en ATENCION ni en PENSANDO. Esa invariante impide el sintoma mas dificil de diagnosticar en vivo: una aplicacion que no se cerro pero se quedo muda para siempre.

El detalle completo esta en `docs/evidencia/T-12-cobertura-de-errores.md`, que incluye tambien lo que no se cubre.

8. Accesibilidad

Los cuatro estados de actividad se distinguen por color y por forma, no solo por color: circulo que late, tres puntos, onda y triangulo. Una persona con daltonismo tiene que poder operar la aplicacion, y si la unica pista fuera el color, para ella la interfaz no comunicaria nada.

Que dos estados compartieran color es un fallo automatico del proyecto, y hay pruebas que lo impiden. Verifican que los cuatro tengan color propio, forma propia y contraste suficiente contra su relleno y contra el fondo de la ventana.

9. Verificacion

Hay 189 pruebas automaticas que corren en unos doce segundos, sin red, sin microfono y sin gastar saldo de API. El historial son 74 commits, uno por tarea.

Las pruebas mas utiles del proyecto no comprueban comportamientos sino prohibiciones: que el orquestador no importe la interfaz, que `tools/` no contenga `eval`, que dos estados no compartan color. Un comportamiento correcto puede volver a romperse; una prohibicion verificada, no.

9.1 Lo que las pruebas no vieron

Es el hallazgo mas util del proyecto y por eso lo pongo aqui en vez de callarlo. Varios defectos reales no los encontro ninguna prueba automatica, sino usar la aplicacion:

Whisper respondia "gracias" ante el silencio, porque alucina con audio vacio. El verde menta se veia azul en pantalla: como decoracion los pasteles funcionan, como senal no. El estado de aviso se emitia correctamente y aun asi nadie lo veia, primero porque duraba microsegundos y despues porque ocurría en una columna distinta de donde yo estaba mirando. El README declaraba un modelo que la aplicacion ya no usaba. Y `--kiosk=url` abria el navegador en su pagina de inicio, con un comando bien formado que significaba otra cosa para el programa que lo recibia.

Todos viven en la frontera entre el codigo y lo que percibe una persona. Ninguna prueba los habria detectado, porque en todos los casos el programa hacia exactamente lo que decia hacer.

10. Modelos y proveedores

Etapa	Proveedor	Modelo	Costo
Transcripcion	Together AI	<code>openai/whisper-large-v3</code>	\$0.0015 por minuto
LLM predeterminado	Together AI	<code>Qwen/Qwen3.8-2.4T-A95B</code>	\$2.50 / \$6.25 por millon
LLM alterno	Together AI	<code>meta-llama/Llama-3.3-70B-Instruct-Turbo</code>	\$1.04 / \$1.04 por millon
Sintesis de voz	Microsoft	<code>edge-tts</code> , voz <code>es-AR-ElenaNeural</code>	sin costo
Tokenizacion (laboratorio)	Hugging Face	tokenizador de <code>Qwen/Qwen2.5-7B-Instruct</code>	sin costo
Embeddings y atencion	Hugging Face	<code>dccuchile/bert-base-spanish-wwm-cased</code>	sin costo
Voz a texto sin internet	local	<code>faster-whisper base</code>	sin costo
LLM sin internet	local	<code>Qwen/Qwen2.5-0.5B-Instruct</code>	sin costo

No entrene ninguno. Todos son preentrenados, como exige el enunciado.
Los elegi probandolos uno por uno contra la API, no leyendo el catalogo, por el motivo que explico en la seccion 6.

11. Consideraciones eticas

El system prompt declara que Elena es una inteligencia artificial y que sus respuestas pueden contener errores, y la aplicacion lo repite en su primer mensaje al abrirse. No se presenta como una persona.

Ninguna credencial vive en el repositorio. La clave de API se lee de un archivo `.env` excluido de Git, y los mensajes de error filtran cualquier rastro de ella antes de mostrarse, porque el repositorio es publico y una captura de pantalla podria acabar en este informe o en el video.

La voz nunca se escribe en disco. Se graba en memoria y desaparece cuando nadie la referencia. Es un dato personal y no se conserva.

Las paginas que puede abrir salen de una lista cerrada, y el dominio real de la direccion se valida antes de construir el comando.

Sobre el arte de la interfaz: el repositorio es publico, asi que no subi ninguna imagen de terceros. Todo lo que se ve esta dibujado por codigo.